



Getting Started

Setup, Tools & Readiness Checklist

ADI MCU Hackathon

David Brogno • Thomas Harper

June 2026

Contents

Getting Started	2
1. What You’re Building (the 2-minute version)	2
2. Prerequisites (knowledge)	3
3. What You Need	4
4. Environment Setup (do this before day one)	4
5. What’s in the Toolbox	5
6. Readiness Checklist	6
7. All Set	6

Getting Started

Read this before the ADI MCU Hackathon starts. This guide gets your machine and your team ready so that on day one you can jump straight into designing your CPU instead of fighting with tool installs. Work through the [Readiness Checklist](#) at the end – if every box is ticked, you are ready.

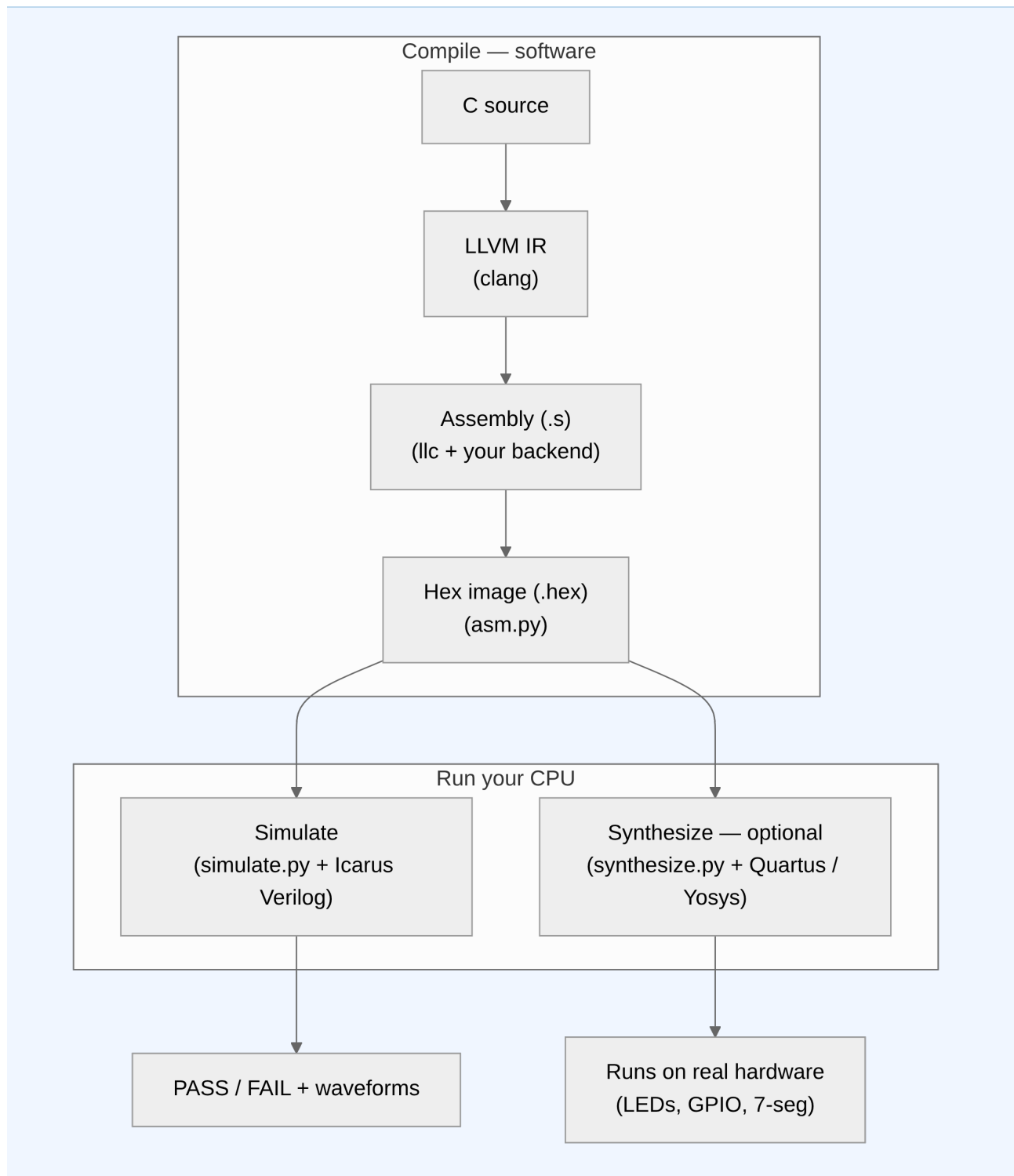
For the hands-on build instructions, see the **Tutorial Workbook** (Stages 0–3 + Stretch Goals). For a narrative design walkthrough with code templates, see the **Project Walkthrough**.

1. What You're Building (the 2-minute version)

Over the ADI MCU Hackathon your team will design a small **CPU** and the software toolchain that targets it:

1. **Define an ISA** – the contract between hardware and software.
2. **Implement it in Verilog** – the hardware that executes those instructions.
3. **Write a compiler backend** – so C code can target your CPU.
4. **Verify it** – in simulation, and optionally on real FPGA hardware.

By the end you will have a complete path from C source to a running program:



The tutorials include worked **examples** at each step to illustrate the choices — but the goal is to design and build your **own** ISA.

2. Prerequisites (knowledge)

You do **not** need any prior CPU-design, compiler, or FPGA experience — the tutorial teaches all of that. You **do** need working familiarity with:

Skill	What you should be comfortable with
Verilog	modules, always blocks, wire/reg, buses, simple testbenches
C	functions, pointers, integer types, bitwise operators
Python	reading and lightly editing scripts (you don't need to be an expert)
Command line	running commands, editing files, navigating folders

***On Python:** you'll use it (e.g. the encoder and the build/simulate scripts), but we provide the bulk of the scripts – you mostly fill in or tweak small pieces, so basic familiarity is plenty.*

3. What You Need

- **Your own laptop** running Windows 10/11, macOS, or Linux, with **~30 GB free** for the dev container. More memory helps the LLVM build (16 GB is comfortable), but it isn't strictly required – on 8 GB just keep the LLVM build jobs low.
- **A way to share and submit your work.** Consider this up front – a GitHub repo is preferred, but use whatever your team is comfortable with.
- **(Optional) An FPGA board** (e.g. DE1-SoC / Intel Cyclone V) if you plan to attempt the hardware stretch goals. Simulation needs no hardware.

4. Environment Setup (do this before day one)

Everything you need is packaged in a **dev container**, so you don't install compilers and EDA tools by hand. The recommended path is VS Code + Docker.

Step 1 – Install VS Code

Download and install **Visual Studio Code**: <https://code.visualstudio.com/>

Step 2 – Install Docker

Install **Docker Desktop** (Windows/macOS) or **Docker Engine** (Linux): <https://www.docker.com/products/docker-desktop/>

Start Docker and confirm it runs:

```
docker --version
docker run --rm hello-world
```

***Windows users:** Docker Desktop uses WSL 2. If prompted, install/enable WSL 2 and reopen Docker Desktop before continuing.*

Step 3 – Install the Dev Containers extension

In VS Code, open the Extensions panel and install **Dev Containers**(ms-vscode-remote.remote-containers).

Step 4 – Get the material

Obtain the ADI MCU Hackathon material as directed by your organizers (a download link, a shared drive, or a Git repository – they'll tell you which). Unpack it somewhere convenient. The rest of this guide assumes the project folder is called `hackathon-mcu/`.

Step 5 – Build & open the dev container

1. Open the `hackathon-mcu/` folder in VS Code.
2. Press F1 (or Ctrl/Cmd+Shift+P) → **Dev Containers: Reopen in Container**.
3. The first time, VS Code builds the container – this compiles LLVM 17 from source and takes a little while, so **do this before the event, not on day one**. It's cached after that.

4. Once it reopens inside the container, verify the toolchain in the integrated terminal:

```
clang --version | head -1
iverilog -V 2>&1 | head -1
yosys -V | head -1
verilator --version
python3 -c "import yaml, cocotb; print('PyYAML + cocotb OK')"
```

A version line for each tool means you're ready for the event.

Optional – save your work: If your team wants version control, initialise a Git repo in the project folder and push it to a new (ideally private) GitHub repository. This is entirely optional and not required to complete the ADI MCU Hackathon.

5. What's in the Toolbox

The container includes everything the tutorial uses. You don't need to know these deeply yet — just what each one is for.

Tool	Role
clang / llc / llvm-link / opt / lld (LLVM 17)	Compile C → LLVM IR → your CPU's assembly
Icarus Verilog (iverilog, vvp)	Compile and run your Verilog in simulation
Verilator	Fast Verilog linting / simulation
GTKWave	View simulation waveforms (.vcd) signal-by-signal
Yosys	Open-source logic synthesis
nextpnr / IceStorm	Open-source FPGA place-and-route (iCE40)
Intel Quartus (optional)	Synthesis/bitstream for Intel FPGAs (DE1-SoC)
Python 3 + PyYAML + cocotb	Drives the build/sim scripts and Python testbenches

The helper scripts

We provide these scripts (in `resources/software/scripts/`) — you don't write them. Instead, you write a small **YAML config file** for each, pointing the script at *your* RTL, backend, and test programs:

Script	Purpose
<code>compile.py</code>	C → LLVM IR → target assembly → .hex
<code>simulate.py</code>	Assemble a program, build the RTL, run it, check registers
<code>build_compiler.py</code>	Rebuild llc with your custom LLVM backend registered
<code>synthesize.py</code>	Generate a Quartus project / run FPGA synthesis

Because every team runs the **same scripts** and supplies only their own .yaml configs, we have a **standardized way to build, simulate, and judge everyone's work**. The workbook shows the config format for each — you'll meet them there, so there's nothing to memorize now.

6. Readiness Checklist

Tick every box **before** the ADI MCU Hackathon begins:

- ☐ VS Code installed
 - ☐ Docker installed and `docker run --rm hello-world` succeeds
 - ☐ Dev Containers extension installed
 - ☐ ADI MCU Hackathon material obtained and opened in VS Code
 - ☐ **Reopen in Container** completed at least once (container built, tools verified)
 - ☐ Comfortable with Verilog, C, and a little Python
 - ☐ Team has skimmed Stage 0 of the workbook and the Project Walkthrough
-

7. All Set

That's everything you need before the event. With your container built and the checklist ticked, you're ready to dive into the Tutorial Workbook on day one. See you there!